

This guide provides a structured framework for Level 3 TSS students to execute a school-based Industrial Attachment Program (IAP) project over four weeks. Since the focus is on foundational skills and static web development, the emphasis is on clean design, organized code, and proper documentation.

1. Project Selection Guidelines

To ensure students pick a project that is both achievable and comprehensive, they should follow these criteria:

- **Solve a Real Problem:** Encourage students to look at their local community (e.g., a local coffee shop, a school club, or a small farm).
 - **Static Scope:** The project should focus on **HTML**, **CSS**, and **Basic JavaScript**. Avoid databases or complex back-end logic.
 - **Module Integration:** The topic must allow for graphic design (logos/banners), UI/UX planning (wireframes), and version control.
 - **SMART Goal:** The project must be specific, measurable, achievable, relevant, and time-bound (must be finished in 4 weeks).
-

2. Implementation Roadmap (4-Week Schedule)

Week 1: Requirements & Design

- **Requirements:** Define the project goal and list the pages needed (e.g., Home, About, Services, Contact).
- **UI/UX:** Create a sitemap and low-fidelity wireframes on paper or a digital tool.
- **Graphic Design:** Use Adobe Photoshop or Illustrator to create a professional logo, icons, and a high-fidelity mockup of the homepage.

Week 2: Layout & Version Control

- **Setup:** Initialize a **Git** repository and create the first commit. Push the project to **GitHub**.
- **Website Development:** Build the structure using HTML. Focus on semantic tags (header, nav, section, footer).
- **Styling:** Start CSS styling. Ensure the layout is responsive (looks good on mobile and desktop).

Week 3: Interactivity & Assets

- **JavaScript:** Add basic functionality, such as a mobile navigation menu, an image slider, or form validation.

- **Asset Finalization:** Export all images from Photoshop/Illustrator in web-friendly formats (PNG, WebP, or SVG).
- **Git Workflow:** Commit changes daily with meaningful messages.

Week 4: Testing & Presentation

- **Testing:** Check the website on different browsers (Chrome, Firefox, Edge).
- **Documentation:** Prepare a final report detailing the tools used and the project's purpose.
- **Presentation:** Prepare a 5-10 minute demonstration of the live website and the GitHub repository.

3. Project Examples for Level 3 Students

Project Idea	Description	Key Features
Local Business Landing Page	A site for a local tailor or grocery store.	Photo gallery of products, contact form, and price list.
School Club Portal	A website for a sports team or debate club.	Event calendar, member bios, and registration form.
Personal Portfolio	A site to showcase the student's own skills.	Resume section, skill bars (CSS), and project links.
Simple Quiz App	An interactive educational tool.	Multiple-choice questions using JavaScript logic.
Tourist Guide Page	A site promoting a specific Rwandan district.	Map integration (iframe), history section, and beautiful imagery.

4. Student IAP Checklist

[!IMPORTANT]

Students should check these items off as they progress to ensure they meet the module requirements.

- ☐ **Project Requirements:** Do I have a written list of features?
- ☐ **Graphic Design:** Is my logo original and high-resolution?
- ☐ **UX/UI:** Did I create a wireframe before I started coding?
- ☐ **Website Development:** Is my HTML code clean and properly indented?
- ☐ **JavaScript:** Is there at least one interactive element (e.g., a "Back to Top" button)?
- ☐ **Version Control:** Do I have at least 10 commits on GitHub?

- [] **Responsiveness:** Does the website work on a smartphone screen?
-

5. Key Points for Success

- **Consistency over Complexity:** It is better to have a perfectly functioning 3-page static website than a broken 10-page site.
- **Code Organization:** Use separate folders for /css, /js, and /images.
- **Documentation:** Students must be able to explain why they chose specific colors or layouts (UX principles).
- **Self-Learning:** Encourage them to use documentation (like W3Schools) when they get stuck with a JavaScript function.